理解 MHA、MQA、GQA 与 MLA

Understanding MHA, MQA, GQA & MLA

中文修订版 · 图表与公式随文排版

原作者：@techNmak · Understanding AI 系列 · https://x.com/techNmak

本版逐段翻译正文、标题、图注和表格，在对应位置保留原始图片与数学公式；图中自然语言标签译为中文。行内公式和独立公式沿用原稿矢量内容，不改写符号、上下标或数值。代码标识符及可执行字符串保留。

中文使用霞鹜文楷（LXGW WenKai），英文和数字使用 Times New Roman。术语括注英文原词。原稿资料与历史陈述按作者原文翻译，不代表重新核查外部资料的最新状态。

专业术语翻译对照表

英文原词	中文译名	缩写／符号
MHA	多头注意力	MHA
MQA	多查询注意力	MQA
GQA	分组查询注意力	GQA
MLA	多头潜在注意力	MLA
Multi-head Attention	多头注意力	MHA
Multi-head Latent Attention	多头潜在注意力	MLA
Grouped-query Attention	分组查询注意力	GQA
Model Capacity	模型容量	
Inference	推理	
KV Cache	键值缓存	KV Cache
Decode	解码	
Query	查询	Q
KV-head Multiplicity	键值头数量	
Representation	表示	
Mental Model	理解框架	
Heads	头	
Query Heads	查询头	
Attention	注意力机制	Attention
Token	词元	
Latent Representation	潜在表示	
Value Heads	值头	
Values	值	
Model	模型	
Keys	键	
Cache Capacity	缓存容量	
Training	训练	
Decoder	解码器	

英文原词	中文译名	缩写／符号
Cache Payload	缓存有效载荷	
Attention Head	注意力头	
Causal Self-attention	因果自注意力	
Tokens	词元	
Transformer	Transformer 模型	Transformer
Queries	查询	
Query Head	查询头	
Batch Size	批大小	
Head Width	头宽度	
Precision	精度	
Activation	激活	
Serving	推理服务	
Incremental Decoding	增量解码	
Key Heads	键头	
Tensor	张量	
KV Heads	键值头	
Batching	批处理	
Tiling	分块	
Multi-query Attention	多查询注意力	MQA
Attention Distribution	注意力分布	
Output Projection	输出投影	
Dense Attention	稠密注意力	
Baseline	基线	
Key/Value Head	键值头	KV Head
Uptraining	追加训练	
Pretraining	预训练	
Pooling	池化	
Models	模型	
Benchmark	基准测试	

英文原词	中文译名	缩写／符号
Decoder-only LLM	仅解码器大语言模型	
Kernel	内核	
Fragmentation	碎片化	
Memory Traffic	内存流量	
Throughput	吞吐量	
Evidence	证据	
Eviction	逐出	
Low-rank Joint Compression	低秩联合压缩	
Hidden State	隐藏状态	
Vector	向量	
Projection Absorption	投影吸收	
RoPE	旋转位置嵌入	RoPE
Low-rank	低秩	
Head Dimension	头维度	
Compressed Latent	压缩潜在表示	
Matrix Multiplication	矩阵乘法	
Latent Compression	潜在压缩	
Up-projection	升维投影	
Operator	算子	
Matrix	矩阵	
Decoupled RoPE	解耦 RoPE	
Positional Encoding	位置编码	
Residual Stream	残差流	
FlashAttention	FlashAttention（方法名保留）	
Softmax	Softmax 归一化	Softmax
Combine	合并	
PagedAttention	分页注意力	
Quantization	量化	
Tensor Shape	张量形状	

英文原词	中文译名	缩写／符号
Fused Kernel	融合内核	
Dtype	数据类型	
Mask	掩码	
Quantized	量化后的	
Allocator Overhead	分配器开销	
Latency	延迟	
FLOPs	浮点运算次数	
Memory Footprint	内存占用	
Mixture-of-Experts	混合专家	MoE
Rotary Position Embedding	旋转位置嵌入	RoPE
Exact Attention	精确注意力	
Large Language Model	大语言模型	LLM

目录导航

封面与说明 · 第 1 页

专业术语翻译对照表 · 第 2 页

理解 MHA、MQA、GQA 与 MLA · 第 8 页

如何阅读本手册 · 第 9 页

1. 从一个注意力头（Attention Head）开始 · 第 10 页

2. 为什么存在多个头 · 第 11 页

3.多头注意力（Multi-head Attention）（MHA）· 第 12 页

4. 训练（Training）和解码对资源的压力不同 · 第 13 页

5. KV-cache公式，推导 · 第 14 页

6. 解码（Decode）带宽：为什么更少的 KV 头有帮助 · 第 15 页

7.多查询注意力（Multi-query Attention）（MQA）· 第 16 页

8. MQA 保存了什么以及不保存什么 · 第 17 页

9.分组查询注意力（Grouped-query Attention）（GQA）· 第 18 页

10. MHA、GQA、MQA 一张图 · 第 19 页

11. GQA 也是一个提升配方 · 第 20 页

12. 原始 GQA 结果实际上显示了什么 · 第 21 页

13. 32Q / 8KV 分组示例 · 第 22 页

14.工作缓存计算：MHA vs GQA vs MQA · 第 23 页

15. 头共享的变化大于内存容量的变化 · 第 24 页

16. 现代型号使GQA成为主流 · 第 25 页

17. MLA 是设计空间的不同分支 · 第 26 页

18.低秩（Low-rank）联合K/V压缩 · 第 27 页

19.吸收技巧：不要仅仅为了缓存而重建完整的K/V · 第 28 页

20.查询（Query）压缩是一个单独的想法 · 第 29 页

21.为什么普通RoPE会破坏简单的MLA吸收 · 第 30 页

22. 解耦绳索 · 第 31 页

23. MLA实际缓存了什么 · 第 32 页

24. DeepSeek-V2的具体MLA尺寸 · 第 33 页

25. 为什么论文说 MLA 就像“GQA 有 2.25 个组” · 第 34 页

26. MHA/GQA/MQA/MLA 缓存比较说明 · 第 35 页

27. 93.3% DeepSeek-V2 数字，范围正确 · 第 36 页

28. DeepSeek 的消融做什么和不做什么 · 第 37 页

29. 四种并排的设计 · 第 38 页

30. FlashAttention 为独立执行轴 · 第 40 页

31. PagedAttention是另一个不同的轴 · 第 41 页

32.当前PyTorch GQA行为 · 第 42 页

33. 如何读取模型（Model）配置 · 第 43 页

34. 常见错误 · 第 44 页

35. 实用的决策框架 · 第 45 页

36. 与理解人工智能系列其余部分的联系 · 第 46 页

参考文献和进一步阅读 · 第 47 页

参考图：针对哪个问题阅读什么内容 · 第 49 页

对应原稿第 1 页

了解 MHA、MQA、GQA 和 MLA

从多头注意力（Multi-head Attention）到潜在 KV 压缩

关于查询（Query）、键和值头结构如何改变 键值缓存（KV Cache）、解码（Decode）带宽、模型容量（Model Capacity）和现代 LLM 推理（Inference）的技术手册 - 从标准多头注意力（Multi-head Attention）到多查询和分组查询注意力（Grouped-query Attention），再到 DeepSeek 的多头潜在注意力（Multi-head Latent Attention）。

如何阅读本手册

对应原稿第 2 页

MHA、MQA、GQA 和 MLA 通常在图中表示为四个框。这种观点很有用，但它隐藏了这些设计存在的原因。重要的问题不仅仅是抽取了多少个头。他们是：

1. 有多少个查询头（Query Heads）产生不同的注意力机制（Attention）分布？

2. 必须为每个先前的词元（Token）存储多少个不同的键/值表示？

3. 在自回归解码过程中实际缓存了什么表示（Representation）？

对于 MHA、MQA 和 GQA，最干净的理解框架（Mental Model）是将查询头（Query Heads）的数量与键/值头（Value Heads）的数量分开。对于 MLA，该模型（Model）不再足够：主要的节省来自缓存低维潜在表示（Latent Representation），而不是普通的完整每头键和值。[1][2][3][4]

> 中央区别。 MHA、GQA和MQA主要可以通过KV头多重性来描述。 MLA 更改缓存的表示（Representation）本身。因此，称 MLA “GQA 的头更少”是一种错误的理解框架（Mental Model）。

本手册重点介绍解码器（Decoder）式自回归推理（Inference），因为持久 KV 缓存容量（Cache Capacity）和重复缓存读取变得尤为重要。当区别很重要时，训练（Training）行为将单独讨论。

符号

我们使用 L表示注意层的数量， T表示缓存序列长度， b表示每个缓存的字节数

标量， H_Q表示查询（Query）头计数， H_{KV}表示键/值头计数， d_h表示普通每头键/值宽度，除非另有说明。

对于具有相同键和值宽度的 MHA/MQA/GQA，一个序列的缓存负载为

$$M_{\mathrm{KV}} = 2LTH_{KV}d_h b.$$

因数二用于键和值。这是一个容量公式，而不是有关物理 HBM 流量的通用表述。

1. 从一个注意力头（Attention Head）开始

对应原稿第 3 页

对于解码（Decode）步骤 t,的一个因果自注意力（Causal Self-attention）头，将新查询 $\boldsymbol{q}_t$与位置$1,\ldots,t.$的键进行比较，如果$\boldsymbol{k}_j,\boldsymbol{v}_j\in\mathbb{R}^{d_h}$,则输出为

$$\boldsymbol{o}_t=\sum_{j=1}^{t}p_{tj}\boldsymbol{v}_j,\qquad p_{tj}=\mathrm{softmax}_j\left(\frac{\boldsymbol{q}_t^\top\boldsymbol{k}_j}{\sqrt{d_h}}\right).$$

重要的推理（Inference）细节是时间不对称。新词元（Tokens）的查询（Query）仅使用一次。当生成后续词元时，将再次需要先前词元的键和值。这就是解码器（Decoder）推理将先前的 K/V 张量存储在 键值缓存（KV Cache）中的原因。

如果没有缓存，解码器（Decoder）将重复重新计算整个前缀的键和值。与

缓存时，模型（Model）计算一次新词元（Token）的 K/V，将它们附加到缓存中，并在后续步骤中重用存储的历史记录。

> 键值缓存（KV Cache）并不是外界关注的可选“速度技巧”。这是在自回归解码期间避免重新计算过去的键/值投影的标准方法。因此，当上下文和批次增长时，它的大小就成为架构问题。

下一个问题是当注意力机制（Attention）有多个头时会发生什么。

2. 为什么存在多个头

对应原稿第 4 页

最初的 Transformer 并行应用多个注意力机制（Attention）头。每个头接收自己的投影查询（Queries）、键和值，计算注意力机制结果，并且头输出被连接并投影回模型（Model）空间。[1]

在常见的等宽情况下，

$$\boldsymbol{q}_i = W_i^Q \boldsymbol{h}, \qquad \boldsymbol{k}_i = W_i^K \boldsymbol{h}, \qquad \boldsymbol{v}_i = W_i^V \boldsymbol{h},$$

对于 head $i \in \{1, \ldots, H\}$.每个头可以学习不同的查询（Query）键兼容性函数和不同的值投影。

通常的解释是“不同的头脑关注不同的事情”是直观的但不完整的。从架构上来说，单独的 K/V 头也意味着解码期间单独的 K/V 投影参数和单独的缓存 K/V 向量。

This second consequence is what motivates MQA and GQA.他们主要不问多个查询头（Query Heads）是否有用。 They ask whether every query head must own a distinct key and value representation for every 词元（Token） in the prefix.

> 不要将人数与模型（Model）宽度混为一谈。在许多架构中，$d_{\text{model}} = H d_h$.这是一种常见的设计选择，而不是数学定律。例如，DeepSeek-V2 的 MLA 配置使用 128 个宽度为 128 的注意力机制（Attention）头，而隐藏维度为5120.[4]

3.多头注意力（Multi-head Attention）（MHA）

对应原稿第 5 页

在标准 MHA 中，查询（Query）头、键头和值头的数量相同：

$$H_Q = H_{KV} = H.$$

For every query head i,there is a corresponding key head and value head.因此，在解码过程中，每个先前的词元（Token）都会将 H键向量和 H值向量贡献给每个注意力机制（Attention）层的缓存。

所有 L层的紧凑的每个词元（Token）缓存计数为

$N_{\text{MHA}} = 2LHd_h$

标量。存储宽度为 b字节，

$M_{\text{MHA}} = 2LTHd_hb.$

这种对 T的线性依赖性很容易被低估。缓存随着序列长度、批大小（Batch Size）、层数、KV 头数、头宽度（Head Width）和存储精度（Precision）而增长。模型（Model）权重可以很好地适应，而缓存仍然限制并发长上下文请求的数量。

MHA 仍然是一个重要的参考点，因为它为每个查询头（Query Head）保留独立的 K/V 投影。但正是这种表征的丰富性才使得它的缓存变得很大。

4. 训练（Training）和解码对资源的压力不同

对应原稿第 6 页

在全序列训练（Training）期间，所有词元（Token）位置都是并行处理的。不存在必须在单词元解码（Decode）迭代中生存的长期推理服务（Serving）缓存。注意力机制（Attention）仍然消耗激活（Activation）内存和带宽，但持久的 KV 缓存问题特别是推理（Inference）问题。

在自回归解码过程中，每个步骤仅为每个序列创建一个新的查询（Query）位置，而模型（Model）会重复关注不断增长的前缀。先前计算的 K/V 张量必须可用于每个步骤。 Shazeer 的 MQA 论文将这些大型 K/V 张量的重复加载确定为增量解码（Incremental Decoding）的主要内存带宽成本。[2]

这就是为什么 MQA/GQA 最好被理解为面向推理（Inference）的架构变化。它们还可以更改训练（Training）参数计数和投影工作，但其标志性好处是生成和缓存的不同 K/V 向量更少。

> 解码（Decode）理解框架（Mental Model）。查询（Query）工作主要是“现在有新词元（Tokens）”。 KV 存储是“所有先前的词元、每一层、每个活动序列”。减少持久的 K/V 表示（Representation）会攻击上下文长度和批大小（Batch Size）累积的部分。

5. KV-cache公式，推导

对应原稿第 7 页

对于一层的一个词元（Token），假设有 H_{KV}键头（Key Heads）和 H_{KV}值头（Value Heads），每个头的宽度

$$d_h.$$

键贡献 $H_{KV}d_h$标量。价值观贡献了另一个 $H_{KV}d_h$.因此：

每层每个词元（Token）的标量 = $2H_{KV}d_h$.

跨 L层和 T缓存词元（Token）位置：

$$N_{\text{KV}} = 2LTH_{KV}d_h.$$

每个缓存标量有 b字节：

$$\underline{M_{\text{KV}} = 2LTH_{KV}d_hb.}$$

对于缓存长度不等的批次 $T_1, \ldots, T_B$,，理想的有效载荷计数为

$$M_{\text{KV}} = 2LH_{KV}d_hb\sum_{r=1}^{B} T_r.$$

实际的推理服务（Serving）系统可能会保留块、填充、页面、量化、卸载或维护元数据，因此分配的内存可能与此张量（Tensor）有效载荷计算不同。

这个公式是架构和系统行为之间的桥梁：MHA、GQA 和 MQA 更改 H_{KV}，同时通常保持 H_Q固定。

6. 解码（Decode）带宽：为什么更少的 KV 头有帮助

对应原稿第 8 页

对于解码（Decode）步骤中的一个层，简单的有效载荷模型（Model）与以下量成正比：

$$2TH_{KV}d_hb,$$

因为新查询（Query）必须与缓存前缀中的键和值进行交互。这不是通用的 HBM 字节计数：真正的内核利用缓存层次结构、平铺、批处理（Batching）、矢量化和重用。但它暴露了架构杠杆。

如果 32 个查询头（Query Heads）各自拥有不同的 K/V 头，则缓存包含每个词元（Token） 32 K 头和 32 V 头。如果相同的 32 个查询头仅共享 8 K/V 头，则持久缓存有效载荷（Cache Payload）将是四分之一。如果所有 32 个查询（Queries）共享一个 K/V 头，则在相同的 L, T, d_h, b 假设下，它的大小相当于 MHA 缓存的三十秒。

注意力机制（Attention）分数算术则不同。仍然有 32 个查询头（Query Heads）产生 32 个注意力机制输出。共享 K/V 头不会删除那些查询头计算。它主要减少 K/V 投影多重性、缓存容量（Cache Capacity）以及需要提供注意力机制的不同缓存 K/V 数据量。

> 这就是为什么“GQA 将注意力机制（Attention）复杂度降低了 4 倍”通常过于模糊而无用。哪种复杂性？在 32Q/8KV 示例中，K/V 投影工作和缓存有效载荷（Cache Payload）可能会下降 4 倍，而密集查询（Query）键评分仍然有 32 个查询头（Query Heads）。

7.多查询注意力（Multi-query Attention）（MQA）

对应原稿第 9 页

多查询注意力（Multi-query Attention）机制保留多个查询头（Query Heads），但在所有查询头之间共享一个键头和一个值头。 Shazeer 引入该变体专门是为了减少增量解码（Incremental Decoding）的 K/V 张量（Tensor）大小和内存带宽要求。[2]

定义计数是

$$H_Q = H, \qquad H_{KV} = 1.$$

每个查询头（Query Head）仍然有自己的查询（Query）投影，因此可以产生不同的注意力分布（Attention Distribution）。共享的是键和值端的前缀表示（Representation）。

用于查询（Query）head i,

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j \left(\frac{\boldsymbol{q}_{t,i}^\top \boldsymbol{k}_j}{\sqrt{d_h}} \right) \boldsymbol{v}_j.$$

每个查询头（Query Head）都会读取相同的 $\boldsymbol{k}_j$和 $\boldsymbol{v}_j$。

> MQA 并不意味着一个注意力头（Attention Head）。表示一对KV头。该模型（Model）仍然可以有许多独立的查询头（Query Heads）和许多不同的注意力机制（Attention）分布。

这种区别是 MQA 周围最常见的实现和解释错误之一。

8. MQA 保存了什么以及不保存什么

对应原稿第 10 页

MHA/MQA缓存公式下，一改替换 HKV头

$$2LTHd_hb \quad \longrightarrow \quad 2LTd_hb.$$

如果所有其他项保持相等，则缓存缩减因子为 H。

MQA 还缩小了 K 和 V 投影输出宽度，通常还缩小了它们的参数计数。然而：

• 查询头（Query Heads）的数量不需要改变；

• 在输出投影（Output Projection）之前，每个查询头（Query Head）的输出仍然包含一个结果；

• 稠密注意力（Dense Attention）仍然将每个查询头（Query Head）与关注的词元（Token）位置进行比较；

• 序列长度依赖性不会神奇地消除。

最初的 MQA 论文在其实验设置中报告了更快的解码速度，相对于其基线（Baseline），质量仅略有下降。[2]该经验结果不应推广为 MQA 始终是质量中性的定理。后来的工作，包括 GQA 和 DeepSeek-V2 研究，提供了一些示例，其中在测试设置中，与 MHA 相比，更积极的 KV 共享会损失质量。[3][4]

因此，该架构暴露了一个真正的权衡：极端的 K/V 共享可以非常高效地缓存，但它也减少了查询头（Query Heads）可用的不同学习 K/V 子空间的数量。

9.分组查询注意力（Grouped-query Attention）（GQA）

对应原稿第 11 页

GQA 在 MHA 和 MQA 之间插入中间点。它使用一些中间编号 G的KV头，而不是 HKV头或一个KV头。查询头（Query Heads）分为几组，一组中的所有查询（Queries）共享该组的键和值头。[3]

让

$$H_Q = H, \qquad H_{KV} = G, \qquad 1 < G < H,$$

并假设 H可被 G.整除，则每个KV组的查询头（Query Heads）数为

$$r = \frac{H}{G}.$$

一个简单的组图是

$$\left\lfloor \frac{i-1}{r} \right.$$

$$g(i) = 1 + \left\lfloor \frac{i-1}{} \right\rfloor,$$

所以查询（Query） head i使用键/值 head $g(i)$.参加

输出变为

$$\boldsymbol{o}_{t,i} = \sum_{j \le t} \mathrm{softmax}_j \left(\frac{\boldsymbol{q}_{t,i}^{\top} \boldsymbol{k}_{j,g(i)}}{\sqrt{d_h}} \right) \boldsymbol{v}_{j,g(i)}.$$

这使得连续统精确：

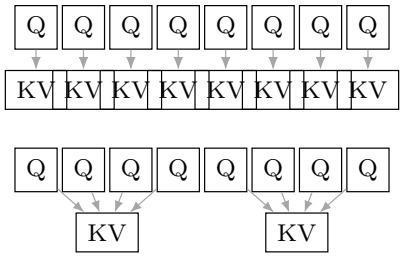
$G = H \Rightarrow$MHA,$1 < G < H \Rightarrow$GQA,$G = 1 \Rightarrow$MQA.

10. MHA、GQA、MQA 一张图

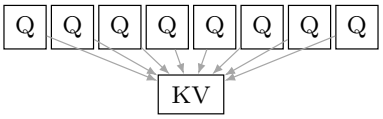
对应原稿第 12 页

MHA：8Q / 8KV

GQA：8Q/2KV



MQA：8Q/1KV



该图特意为成对的键/值头绘制了一个“KV”框。所有三个示例中 Q 框的数量均保持不变。仅不同 KV 对的数量发生变化。

因此，对于固定的 d_h，缓存有效载荷（Cache Payload）分别与 8、2 和 1 成比例。但在每种情况下，查询（Query）头输出的数量仍然是八个。

这张图是读取现代模型（Model）配置的最简单可靠的方法：分别识别查询（Query）头计数和 KV 头计数。

11. GQA 也是一个提升配方

对应原稿第 13 页

GQA 论文不仅仅命名了中间架构。它询问是否可以将现有的 MHA 检查点转换为 MQA 或 GQA，而无需从头开始重新训练。[3]

为了进行转换，对每个目标组内的 K/V 头进行均值池化以初始化共享 K/V 头。然后将转换后的模型（Model）继续进行预训练（Pretraining）。该论文的标题实验仅在这个上训练阶段使用了原始预训练计算的 5%。[3]

这个历史细节很重要，因为有两个想法经常被推翻：

• GQA 作为架构：KV 头少于查询头（Query Heads）。

• GQA 上行训练：将 MHA 检查点转换为该架构的特殊方法。

可以使用 GQA 从头开始训练模型（Model）；它不必源自 MHA 检查点。相反，均值池初始化是转换方法的一部分，而不是分组查询注意力（Grouped-query Attention）的数学定义。

当前的 Hugging Face Llama 配置文档仍然描述了这种均值池转换约定，用于从 MHA 检查点构造分组 K/V 头。[7]

12. 原始 GQA 结果实际上显示了什么

对应原稿第 14 页

GQA 论文中被引用最多的结果来自 T5-XXL 模型（Models），这些模型经过转换和上训练，仅占原始预训练（Pretraining）计算量的 5%。在该设置中，作者报告：[3]

型号	时间/样本（秒）	Avg. score	解读
MHA-XXL	1.51	47.2	质量参考
MQA-XXL	0.24	46.6	这三个中最快的
GQA-8-XXL	0.28	47.1	near MQA time, near MHA average

平均。分数

接近 MQA 时间，接近 MHA 平均值

平均值结合了论文的摘要、翻译、问答评价集；指标因任务而异。上表是对论文报告结果的故意缩减选择，而不是新的通用基准。

正确的结论很窄：在该实验中，GQA-8 恢复了 MHA 基线（Baseline）的大部分平均质量，同时保留了接近 MQA 的推理（Inference）时间。该论文并未证明每个仅解码器的 LLM、内核（Kernel）、批大小（Batch Size）或硬件平台都会表现出相同的质量-速度曲线。

这种范围阅读比“GQA 以 MQA 的速度提供 MHA 的质量”这一口号更有用。

13. 32Q / 8KV 分组示例

对应原稿第 15 页

假设一个层有

$$H_Q = 32, \qquad H_{KV} = 8, \qquad d_h = 128.$$

然后各个KV头服务

$$r = \frac{32}{8} = 4$$

查询头（Query Heads）。

一个自然的分组是：

$$\{Q_1, Q_2, Q_3, Q_4\} \to KV_1,$$
$$\{Q_5, Q_6, Q_7, Q_8\} \to KV_2,$$

依此类推通过 KV_8.

对于一层的一个词元（Token）：

• 查询（Query）表示（Representation）宽度：$32 \times 128 = 4096$标量；

• 键缓存宽度：$8 \times 128 = 1024$标量；

• 值缓存宽度：$8 \times 128 = 1024$标量；

• 总缓存K/V 宽度：2048 个标量。

在具有 32 KV 头的 MHA 下，缓存宽度将为 8192 标量。因此，在此示例中，GQA 将原始 K/V 缓存负载减少了 4 倍，同时保留了 32 个查询头（Query Heads）。

最后一句话是重点：模型（Model）仍然有 32 个查询头（Query Heads）。 GQA 更改这些查询（Queries）参考的不同 K/V 表示的数量。

14.工作缓存计算：MHA vs GQA vs MQA

对应原稿第 16 页

在所有变体中使用相同的配置：

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2 \text{ bytes}.$$

MHA: H_{KV}= 32

$$M = 2 \cdot 32 \cdot 8192 \cdot 32 \cdot 128 \cdot 2 = 4{,}294{,}967{,}296 \text{ bytes} = 4 \text{ GiB}.$$

GQA: H_{KV}= 8

$$M = 1{,}073{,}741{,}824 \text{ bytes} = 1 \text{ GiB}.$$

MQA: H_{KV}= 1

$$M = 134{,}217{,}728 \text{ bytes} = 128 \text{ MiB}.$$

变体	KV头	缓存	相对于MHA
MHA	32	4GiB	1x
GQA	8	1GiB	1/4
MQA	1	128 MiB	1/32

128 兆字节

这些是一个序列的张量（Tensor）有效载荷数。它们排除分配器元数据、碎片、其他激活、模型（Model）权重、临时工作空间和特定于框架的存储。

15. 头共享的变化大于内存容量的变化

对应原稿第 17 页

减少 H_{KV}会影响模型（Model）的几个部分：

K/V 投影宽度。在传统实现中，较少的 KV 头意味着较窄的 K 和 V 投影输出。因此，当头部宽度固定时，它们的参数计数和投影工作可以大致与 H_{KV}成比例下降。

缓存容量（Cache Capacity）。每个词元（Token）的持久缓存与 H_{KV}·线性扩展

解码（Decode）期间读取不同的缓存有效载荷（Cache Payload）。更少的唯一 KV 向量可以减少必须从内存提供的 K/V 数据量，特别是当内核在多个查询（Queries）中重用共享 KV 头时。

代表性分享。多个查询头（Query Heads）现在询问相同的 K/V 表示（Representation）。这是一个架构约束，而不仅仅是系统优化。

不会自动下降相同因素的是完整的稠密注意力（Dense Attention）算法。每个

查询头（Query Head）仍然形成自己的注意力机制（Attention）分数和加权输出。确切的运行时影响取决于预测、注意力机制数学、内存流量（Memory Traffic）、内核（Kernel）设计、批大小（Batch Size）和硬件的相对成本。

16. 现代型号使GQA成为主流

对应原稿第 18 页

GQA 迅速从转换技术转变为标准模型（Models）设计选择。 Llama 2 在其较大的 34B 和 70B 模型中使用了 GQA，以提高推理（Inference）可扩展性。[8]Mistral 7B 还明确采用了 GQA 以实现更快的推理。然后，[9]Meta 在 8B 和 70B Llama 3 上使用了 GQA尺寸.[10]

这些例子与其说是一场流行竞赛，不如说是设计压力的证据（Evidence）：长上下文、更大批量和高吞吐量（Throughput）推理服务（Serving）使得在每个查询头（Query Head）复制 K/V 状态的成本变得越来越明显。

尽管如此，“现代 LLM = GQA”还是太宽泛了。架构继续探索其他策略，包括潜在缓存压缩（例如 MLA）以及其他形式的 KV 压缩或驱逐。重要的教训是，KV 表示（Representation）现在是一流的架构决策。

> 模型（Models）示例是实现事实，而不是定义。无论哪个指定型号采用它，GQA 仍然是 GQA，并且型号系列可以跨尺寸或跨代改变注意力机制（Attention）设计。

17. MLA 是设计空间的不同分支

对应原稿第 19 页

MQA 和 GQA 询问：

应存储多少个常规 K/V 头？

多头潜在注意力（Multi-head Latent Attention）提出了一个不同的问题：

我们是否需要存储传统的每头 K/V 张量？

DeepSeek-V2 引入了具有低秩联合压缩（Low-rank Joint Compression）键和值的 MLA。隐藏状态（Hidden State）不是为每个头和词元（Token）缓存完整的键向量（Vector）和完整的值向量，而是被投影到一个紧凑的潜在向量中，从中导出内容 K/V 表示。[4]

这就是为什么 MLA 不应放置在标记为“KV 头数”的简单一维轴上。其主要压缩参数是潜在宽度 d_c，而不是 H_{KV}.

该设计还有一个单独的小位置关键组件，因为普通 RoPE 会干扰代数投影吸收（Projection Absorption），从而使潜在缓存变得高效。这种交互对于理解 MLA 来说至关重要，而不是实现脚注。

18.低秩（Low-rank）联合K/V压缩

对应原稿第 20 页

设 $\boldsymbol{h}_t \in \mathbb{R}^{d_{\text{model}}}$为token t.的层输入，MLA首先将其压缩为共享的K/V潜伏

$$\boldsymbol{c}_t^{KV} = W_D^{KV}\boldsymbol{h}_t, \qquad \boldsymbol{c}_t^{KV} \in \mathbb{R}^{d_c}.$$

然后可以从该潜在内容生成内容键和值：

$$\boldsymbol{k}_t^C = W_U^K \boldsymbol{c}_t^{KV}, \qquad \boldsymbol{v}_t^C = W_U^V \boldsymbol{c}_t^{KV}.$$

DeepSeek-V2使用的 d_c比全串联多头K或V宽度小很多。[4]

短语“低秩（Low-rank）”应该相对于大的完整 K/V 投影空间来解释。潜伏不必小于一个单独的头维度（Head Dimension）。以DeepSeek-V2为例， d_h= 128，而 d_c= 512。重要的是512远低于完整的128头内容宽度$128 \times 128 = 16{,}384$.[4]

在优化推理（Inference）期间，潜在的 $\boldsymbol{c}_t^{KV}$

是可重用的内容状态。这是与普通 MHA/GQA/MQA 缓存的结构突破。

$$\vec{t}$$

19.吸收技巧：不要仅仅为了缓存而重建完整的K/V

对应原稿第 21 页

乍一看，潜在压缩（Latent Compression）似乎仅通过添加解压缩工作来节省存储：每次需要时从 $\boldsymbol{c}^{KV}$重建 $\boldsymbol{k}^{C}$和 $\boldsymbol{v}^{C}$。 MLA 通过矩阵乘法（Matrix Multiplication）的结合性避免了这种朴素的实现。[4]

对于一个头，内容分数包含

$$(\boldsymbol{q}_i^C)^\top (W_{U,i}^K \boldsymbol{c}_j^{KV}).$$

将其重新关联为

$$\left((W_{U,i}^K)^\top \boldsymbol{q}_i^C\right)^\top \boldsymbol{c}_j^{KV}.$$

键向上投影有效地移至查询（Query）端。缓存的前缀词元（Tokens）可以保留在潜在空间中。

对于价值观来说，

$$\sum_j p_{ij}(W_{U,i}^V \boldsymbol{c}_j^{KV}) = W_{U,i}^V \left(\sum_j p_{ij} \boldsymbol{c}_j^{KV}\right).$$

值上投影可以在加权潜在聚合之后应用，并与周围的输出侧线性图相结合。

> 实际的想法不是“压缩，然后完全解压缩每个缓存的词元（Token）”。它重写周围的线性代数，以便注意力机制（Attention）可以与压缩的潜在变量一起运行，同时固定的投影矩阵被吸收到相邻的计算中。

20.查询（Query）压缩是一个单独的想法

对应原稿第 22 页

DeepSeek-V2 还在查询（Query）路径上应用低秩（Low-rank）投影：

$$\boldsymbol{c}_t^Q = W_D^Q \boldsymbol{h}_t, \qquad \boldsymbol{q}_t^C = W_U^Q \boldsymbol{c}_t^Q.$$

该论文的目的是减少训练（Training）期间的激活（Activation）记忆。它明确指出查询（Query）压缩不会减少推理（Inference）键值缓存（KV Cache）。[4]

这很直观。查询（Queries）是针对当前词元（Token）生成的，而不是未来解码（Decode）步骤需要保留的持久历史记录。压缩查询路径可以改变训练（Training）激活（Activation）存储和参数化，但长期推理服务（Serving）缓存位于键/值侧。

这种区别在 MLA 之外很有用：在评估任何“注意力机制（Attention）压缩”声明时，询问正在压缩的内容以及该张量（Tensor）何时存在？较小的临时查询（Query）激活（Activation）

更小的持久键值缓存（KV Cache）解决了不同的系统问题。

21.为什么普通RoPE会破坏简单的MLA吸收

对应原稿第 23 页

RoPE 对查询（Queries）和密钥应用位置相关的轮换。在普通注意力机制（Attention）中，这正是我们想要的：查询键兼容性取决于通过旋转的相对位置。但是在线性投影之间插入的位置相关运算符会更改 MLA.[5]可用的代数

如果没有位置旋转，内容分数可以重新关联，因此固定的键向上投影

移动到查询（Query）端。使用重建内容密钥上的 RoPE，分数示意性地包含查询侧向量（Vector）和密钥上投影之间的位置相关变换：

$$\boldsymbol{q}_t^\top R_t^\top R_j W_U^K \boldsymbol{c}_j^{KV}.$$

因为 R_j取决于缓存的词元（Token）位置 j,，所以没有单个位置无关的矩阵（Matrix）可以简单地吸收 W_U^K

U适用于所有前缀位置。

DeepSeek-V2 将此描述为标准 RoPE 放置与所需的低阶 KV 吸收之间的不兼容。在每个解码（Decode）步骤中重建整个前缀的特定于位置的完整密钥将破坏大部分推理（Inference）优势。[4]

解决方案是将位置信息保留在单独的较小路径上。

22. 解耦绳索

对应原稿第 24 页

MLA 将键/查询（Query）表示（Representation）分离为内容部分和 RoPE 承载部分。用于查询head i:

$$\boldsymbol{q}_{t,i} = [\boldsymbol{q}^C_{t,i}; \boldsymbol{q}^R_{t,i}],$$

而关键是

$$\boldsymbol{k}_{j,i} = [\boldsymbol{k}^C_{j,i}; \boldsymbol{k}^R_{j}].$$

RoPE查询（Query）组件 $\boldsymbol{q}^R_{t,i}$

j在 DeepSeek-V2 结构中跨头共享。[4]

t,i是头部专用的，但RoPE关键部件 $\boldsymbol{k}^R_i$

因此，注意力机制（Attention）分数结合了内容和位置兼容性：

$$\frac{(\boldsymbol{q}^C_{t,i})^\top \boldsymbol{k}^C_{j,i} + (\boldsymbol{q}^R_{t,i})^\top \boldsymbol{k}^R_{j}}{\sqrt{d_h + d^R_h}}.$$

只有小型共享 RoPE 密钥必须与 K/V 潜在密钥一起添加到持久缓存中。大的内容键可以通过投影吸收（Projection Absorption）保持隐式。

该设计是架构协同设计的一个清晰示例：位置编码（Positional Encoding）的修改并不是因为 RoPE“不好”，而是因为其通常的放置阻碍了架构想要利用的推理（Inference）优化。

23. MLA实际缓存了什么

对应原稿第 25 页

通过解耦 RoPE（Decoupled RoPE），DeepSeek-V2 每层每个词元（Token）缓存两个片段：

1.联合K/V潜伏 $\boldsymbol{c}_t^{KV}$

$$_t^{KV} \in \mathbb{R}^{d_c};$$

$$_t^R \in \mathbb{R}^{d_h^R}.$$

2.共享位置键 $\boldsymbol{k}_t^R$

因此缓存有效载荷（Cache Payload）是

$N_{\text{MLA}} = LT(d_c + d_h^R$

$$_h^R)$$

标量，或

$M_{\text{MLA}} = LT(d_c + d_h^R$

$$\underline{_h^R)b.}$$

请注意相对于普通 K/V 缓存消失的内容：

• 对于单独缓存的完整K 和V 张量，主因数不为2；

• no H_{KV}术语描述了传统KV 头的集合。

这就是为什么将 MLA 强制输入标准公式 $2LTH_{KV}d_hb$在概念上是错误的。缓存对象不同。

对于可变长度批次，理想化的有效载荷可概括为

$$_h^R)b\sum_r T_r.$$

$M_{\text{MLA}} = L(d_c + d_h^R$

推理服务（Serving）分配器和内核在此架构有效载荷之上添加自己的实现细节。

24. DeepSeek-V2的具体MLA尺寸

对应原稿第 26 页

DeepSeek-V2使用60个Transformer层，隐藏维度5120，128个注意力机制（Attention）头，内容头宽度 d_h = 128，KV潜在宽度 d_c= 512，查询（Query）潜在宽度1536，以及解耦的RoPE宽度 d_h^R

$$'^R_h = 64.$$ [4]

重要的比率是

$$d_c = 4d_h, \qquad d_h^R = \frac{d_h}{2}.$$

因此，每层每个词元（Token）的缓存有效载荷为

$$d_c + d_h^R$$

$_h$= $4.5d_h$= 576 个标量.

跨越 60 层：

$$60 \times 576 = 34{,}560$$

每个词元（Token）的标量。这与论文注意力机制（Attention）消融表中大型 MLA 模型（Model）报告的 34.6K 元素数字相匹配。[4]

这里出现了一个微妙但有价值的架构启示：DeepSeek-V2's d_h= 128，并且 128 个头并不意味着 16,384 维的残差流（Residual Stream）。模型（Model）隐藏尺寸为 5120。头部宽度和隐藏宽度是架构参数，而不是每个设计中必须满足 $d_{\text{model}} = Hd_h$的恒等式。

25. 为什么论文说 MLA 就像“GQA 有 2.25 个组”

对应原稿第 27 页

对于普通的GQA，每个词元（Token）的每层缓存包含

$$2Gd_h$$

标量，其中 G是KV组/头的数量。对于 DeepSeek-V2 MLA，它包含

$$d_c + d_h^R = 4.5d_h.$$

等效有效载荷大小：

$$2Gd_h = 4.5d_h \quad \Rightarrow \quad G = 2.25.$$

这是缓存大小的等价，而不是具有 2.25 KV 头的字面架构。人数是离散的。比较表明，DeepSeek-V2 的 MLA 缓存存储的元素与假设的常规 GQA 缓存（具有 2.25 个组）一样多。[4]

这种比较很有用，因为它说明了为什么 MLA 不仅仅是另一个头部共享点。它的缓存可以占用不对应于任何整数常规 键值头 计数的大小，因为它存储的对象是潜在的加上位置组件。

> 不要将“相当于 GQA 具有 2.25 组”变成“MLA 具有 2.25 KV 头”。事实并非如此。

26. MHA/GQA/MQA/MLA 缓存比较说明

对应原稿第 28 页

返回到早期的教育设置：

$$L = 32, \quad T = 8192, \quad H_Q = 32, \quad d_h = 128, \quad b = 2.$$

对于 MLA 大小的示例，请使用 DeepSeek-V2 的比率 d_c= 512 和 d_h^R

h= 64. 那么

$M_{\text{MLA}} = 32 \cdot 8192 \cdot 576 \cdot 2 = 301{,}989{,}888$字节 = 288 MiB.

变体	缓存	vs. MHA	缓存了什么
MHA，32KV	4GiB	1x	32K+32V头
GQA，8KV	1GiB	1/4	8 K + 8 V 头
MQA，1KV	128 MiB	1/32	1K+1V头
MLA 大小的示例	第288章	0.0703x	512-d 潜在 + 64-d RoPE 密钥

128 兆字节

对阵 MHA

一个有用的惊喜是，MQA 的原始缓存容量（Cache Capacity）小于 MLA 参数化。 MLA 的目的不是“总是产生最小的缓存”。它的目标是提供更丰富的学习注意力机制（Attention），同时仍然积极压缩缓存。

这是一个比仅通过记忆对四种方法进行排名要准确得多的理解框架（Mental Model）。

27. 93.3% DeepSeek-V2 数字，范围正确

对应原稿第 29 页

DeepSeek-V2 的摘要报告称，与其早期型号 DeepSeek 67B 相比，键值缓存（KV Cache）减少了 93.3%。[4]

这个数字经常被重复，就好像“MLA 将 键值缓存（KV Cache）减少了 93.3%”。这种概括是不合理的。MLA 缓存减少取决于基准架构和诸如

$$H_{KV},\ d_h,\ d_c,\ d_h^R$$

h，层数和存储精度（Precision）。

人们至少可以进行三种不同的比较：

• 论文中部署的DeepSeek-V2 与DeepSeek 67B；

• MLA 与假设的相同形状的 MHA 架构；

• MLA 与具有选定数量的KV 头的GQA 或MQA 架构相比。

他们产生不同的百分比。论文中93.3%的数字仅属于第一个比较。

> 出版规则。切勿在未指定基线（Baseline）的情况下引用压缩百分比。“93.3%”是 DeepSeek-V2-vs-DeepSeek-67B 结果，而不是 MLA 的常数。

28. DeepSeek 的消融做什么和不做什么

对应原稿第 30 页

DeepSeek-V2 包括 MQA、GQA 与 8 组和 MHA 的受控 7B 密集比较。在四个报告的硬基准测试中，MHA 得分最高，GQA-8 中等，MQA 最低。[4]

一小部分内容说明了这种模式：

基准测试	MQA	GQA-8	MHA
BBH（3 发 EM）	33.2	35.6	37.0
MMLU（5连发）	37.9	41.2	45.2

同一篇论文分别比较了匹配的小型和大型 MoE 设置中的 MLA 和 MHA。 MLA 使用小得多的报告 KV 缓存，并且在大多数列出的基准值上更高；小 MLA 模型（Model）在 C-Eval 上略低，而在 BBH、MMLU 和 CMMLU 上较高，而大 MLA 模型在所有四个列出的基准上都较高。[4]

仔细的解释是经验性的：对于这些架构、数据、训练（Training）配方和评估，DeepSeek 发现 MLA 比比较的替代方案具有更强的缓存质量权衡。 MLA 普遍优于 MHA 或 GQA 并不是一个定理。

29. 四种并排的设计

对应原稿第 31 页

	MHA	GQA	MQA	MLA
Query heads	many	many	many	many

查询头（Query Heads）

许多

许多

许多

许多

传统的 KV 头缓存

每 Q 头 1 个 少于 Q 头

一个潜在的共享对是主要的缓存对象

主缓存杠杆 无 KV 头共享一对共享 KV

低阶联合潜在KV

$2H_Q d_h$	$2H_{KV} d_h$	$2d_h$	$d_c + d_h^R$

每个词元（Token）/层缓存

常驻位置块普通K包含它

普通K包括它

普通K包括它

DeepSeek-V2 中的共享 RoPE 密钥

主要动机独立于每个查询头（Query Head）的 K/V

质量/效率的中间立场

积极的KV共享

压缩缓存容量更丰富

该表突出了两个独立的想法：

头部共享控制存在多少个常规 K/V 子空间。

潜在压缩（Latent Compression）控制存储什么表示（Representation）。

未来的注意力机制（Attention）可以混合或扩展这些轴，因此最持久的技能是读取实际的张量（Tensor）形状并缓存内容，而不是记住四个首字母缩略词。

30. FlashAttention 为独立执行轴

对应原稿第 32 页

FlashAttention 改变了稠密注意力（Dense Attention）的执行计划，以减少注意力机制（Attention）中间体昂贵的内存流量（Memory Traffic），同时计算精确的密集 Softmax 注意力机制。它没有定义一个模型（Model）有多少个KV头。[11]

MHA/MQA/GQA/MLA 是关于 Q/K/V 表示（Representation）的架构选择，特别是在这里，生成和缓存什么 K/V 状态。因此，模型（Model）可以将 GQA 与 FlashAttention 系列内核（Kernel）结合起来。当前的 PyTorch 文档在多个缩放点积注意力机制（Attention）后端中明确支持 GQA，包括记录约束下 CUDA 上的 FlashAttention。[6]

这两个问题是不同的：

• 注意力机制（Attention）架构：存在哪些Q/K/V 张量？

• 注意内核（Kernel）：数学运算在硬件上是如何调度的？

这与“FlashAttention 减少 键值缓存（KV Cache）”具有误导性的原因相同。 FlashAttention 主要避免具体化二次注意力机制（Attention）中间体； MQA/GQA/MLA 攻击持久键/值表示（Representation）。

31. PagedAttention是另一个不同的轴

对应原稿第 33 页

引入PagedAttention来解决LLM推理服务（Serving）中键值缓存（KV Cache）的内存管理：缓存块的碎片、分配、共享和灵活放置。[12]

它没有规定模型（Model）是否使用 MHA、GQA 或 MQA。这些选择决定了逻辑 KV 有效载荷。分页决定了如何在推理服务（Serving）内存系统中管理有效载荷。

MLA 使区别更加清晰，因为它的持久缓存对象不是普通的完整 K/V。推理服务（Serving）引擎仍然需要 MLA 状态的块分配、调度和内存管理，但所管理的表示（Representation）是特定于体系结构的。

因此，一个有用的三轴模型（Model）是：

1.表示（Representation）：MHA / GQA / MQA / MLA。

2. 内核（Kernel）执行：标准关注/FlashAttention 系列内核/其他优化内核。

3.推理服务（Serving）内存管理：连续分配/分页缓存/卸载/量化（Quantization）/前缀重用。

优化一个轴并不会让其他轴消失。

32.当前PyTorch GQA行为

对应原稿第 34 页

截至 2026 年 9 月，torch.nn.function.scaled_dot_product_attention 公开了一个 enable_gqa 选项，并将 GQA 记录为实验性功能。[6]

记录的形状符号将 H_q,查询头（Query Heads）的数量与 H,键/值头（Value Heads）的数量区分开来。当前的限制包括

$$H_K = H_V.$$

$H_q \bmod H = 0,$

该文档包括一个参考等效的 Python 实现，该实现使用 Repeat_interleave 扩展键/值头（Value Heads）。这样做是为了匹配查询（Query）头计数。该片段描述了等效的语义；它不应该被理解为声称每个优化的后端物理上都实现了重复的 K/V 张量。

当前文档还指出了后端特定的支持和限制，这些支持和限制的变化速度比架构数学更快。因此，生产代码应将已安装的框架和内核（Kernel）文档视为设备、数据类型（Dtype）、掩码（Mask）、嵌套张量（Tensor）和 GQA 支持的权威文档。

> 框架支持是特定于版本的。 GQA的数学定义是稳定的；哪个融合内核（Fused Kernel）接受给定的张量形状（Tensor Shape）则不接受。

33. 如何读取模型（Model）配置

对应原稿第 35 页

许多 Transformer 配置公开两个单独的字段：

```
num_attention_heads = Hq
```

num_key_value_heads = Hkv

当前的 Hugging Face Llama 文档规定了通常的解释：[7]

• HKv == Hq ⇒MHA；

• HKv ==1 ⇒MQA；

•1 <Hkv <Hq ⇒GQA。

假设解码器（Decoder）有 48 层、64 个查询头（Query Heads）、8 个 KV 头、头宽度（Head Width） 128 和 BF16 缓存元素。对于 16,384 个词元（Token）的单个序列，

$$M_{\mathrm{KV}} = 2 \cdot 48 \cdot 16384 \cdot 8 \cdot 128 \cdot 2 = 3 \text{ GiB}.$$

该架构是 GQA，每个 KV 头有 8 个查询头（Query Heads）。

不要盲目地将这个配置读取规则应用于MLA。 MLA 需要特定于架构的字段，例如 KV 潜在等级和解耦 RoPE（Decoupled RoPE） 维度，因为传统的 num_key_value_heads 不再描述缓存。

34. 常见错误

对应原稿第 36 页

“MQA 有一个查询头（Query Head）。”错误的。 MQA 在多个查询（Query）中共享一对 K/V 头

头。

“GQA 减少了注意力机制（Attention）头的数量。”通常会产生误导。减少KV头

多重性，同时保留查询（Query）头计数。

“GQA 使稠密注意力（Dense Attention）算术呈线性。”错误的。它改变了 K/V 表示（Representation）并且

可以降低投影/缓存成本；它不会消除密集的词元（Token）交互。

“MLA 只是更具攻击性的 GQA。”错误的。 MLA的定义缓存是低阶联合K/V

DeepSeek 设计中潜伏加上一个小位置键。

“MLA 使用的缓存始终少于 MQA。”错误的。使用 DeepSeek-V2 的$4.5d_h$缓存有效载荷（Cache Payload）

，

等宽 MQA 每层/词元（Token）存储 $2d_h$。 MLA 正在追求不同的质量与容量权衡。

“DeepSeek的93.3%就是MLA的压缩率。”错误的。这是报告的DeepSeek-V2

与 DeepSeek 67B 比较。

“**FlashAttention、GQA 和 PagedAttention** 是相同版本的竞争版本

优化。”错误的。它们在不同的轴上运行：执行计划、表示（Representation）和缓存管理。

“头维度（Head Dimension）必须等于模型（Model）宽度除以头部数量。”常见，但不常见

通用的。

35. 实用的决策框架

对应原稿第 37 页

在阅读或设计注意力机制（Attention）架构时，请按顺序提出以下问题：

1. 存在多少个查询头（Query Heads）？这告诉您产生了多少个不同的查询（Query）端注意通道。

2. 传统KV头有多少种？如果这低于查询（Query）头计数，则模型（Model）在 MQA/GQA 系列中使用 KV 共享。

3. 解码（Decode）过程中到底保留了什么？完整的 K/V 张量、潜在的、量化的状态、选定的词元（Tokens），还是其他什么？

4. 位置如何表示？ RoPE 或其他位置机制可以影响投影重新安排的合法性。

5. 哪些成本实际上下降了？投影 FLOP、缓存容量（Cache Capacity）、缓存流量、注意力机制（Attention）算法、分配器开销（Allocator Overhead）或实际运行延迟（Latency）是不同的指标。

6. 有什么证据（Evidence）支持质量声明？培训设置和基准很重要。

该框架比记住从 MHA 到 MQA 到 GQA 到 MLA 的线性演变更持久。架构不会沿着一个轴发展。它们以不同的组合来交换表示（Representation）容量、内存占用（Memory Footprint）、带宽、算术和实现复杂性。

> 紧凑的理解框架（Mental Model）：MHA 为每个查询头（Query Head）提供了自己的 K/V 对。MQA 在所有查询（Queries）中共享一对。 GQA 在每个查询组中共享一对。 MLA 将存储的对象从完整的每头 K/V 更改为学习的潜在表示（Latent Representation）加上一个小的位置组件。

36. 与理解人工智能系列其余部分的联系

对应原稿第 38 页

本主题位于之前四本手册的交叉点：

注意。 MHA是基线（Baseline）多头机制； MQA/GQA/MLA 在改变查询（Query）/键/值表示的组织方式时保留稠密注意力（Dense Attention）。

键值缓存（KV Cache）。缓存公式使 KV 头多重性的系统后果变得明确。 MLA 然后演示了为什么普通缓存公式是特定于体系结构的而不是通用的。

绳索。 MLA 的解耦 RoPE（Decoupled RoPE） 之所以存在，是因为位置相关的旋转会干扰用于潜在 K/V 缓存的投影吸收（Projection Absorption）。

FlashAttention。 FlashAttention 优化注意力机制（Attention）的运行方式； MQA/GQA/MLA 改变存在哪些注意力机制张量以及保留什么状态。

这些概念共同构成了一个有用的层次结构：

数学注意力 →头部架构 →位置编码

→持久缓存 →内核执行.

将这些层分开是准确推理（Inference）现代 Transformer 推理的最佳方法之一。

参考文献和进一步阅读

对应原稿第 39 页

[1] Vaswani, A. et al. 您所需要的就是注意力机制（Attention）。 NeurIPS 2017.https://arxiv.org/abs/1706.03762

原题名与书目信息：[1] Vaswani, A. et al. Attention Is All You Need. NeurIPS 2017. https://arxiv.org/abs/1706.03762

[2] Shazeer, N. 快速变压器解码：您只需要一个写头。 2019.https://arxiv.org/ 绝对/1911.02150

原题名与书目信息：[2] Shazeer, N. Fast Transformer Decoding: One Write-Head Is All You Need. 2019. https://arxiv.org/ abs/1911.02150

[3] Ainslie, J. et al. GQA：从多头检查点训练（Training）广义多查询（Query）变压器模型（Models）。 EMNLP 2023.https://aclanthology.org/2023.emnlp-main.298/

原题名与书目信息：[3] Ainslie, J. et al. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. EMNLP 2023. https://aclanthology.org/2023.emnlp-main.298/

[4] DeepSeek-AI et al. DeepSeek-V2：强大、经济且高效的专家混合语言模型（Model）。 2024.https://arxiv.org/abs/2405.04434

原题名与书目信息：[4] DeepSeek-AI et al. DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model. 2024. https://arxiv.org/abs/2405.04434

[5] Su, J. et al. RoFormer：具有旋转位置嵌入（Rotary Position Embedding）的增强型变压器。神经计算 2024；预印本于 2021 年首次发布。https://arxiv.org/abs/2104.09864

原题名与书目信息：[5] Su, J. et al. RoFormer: Enhanced Transformer with Rotary Position Embedding. Neurocomputing 2024; preprint first posted 2021. https://arxiv.org/abs/2104.09864

[6] PyTorch. torch.nn.functional.scaled_dot_product_attention documentation. torch.nn.function.scaled_dot_product_attention 文档。当前行为于 2026 年 9 月检查。 https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_ dot_product_attention

原题名与书目信息：[6] PyTorch. torch.nn.functional.scaled_dot_product_attention documentation. Current behavior checked September 2026. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_ dot_product_attention

[7] Hugging Face Transformers. Llama配置文档，包括num_attention_heads和num_key_value_heads。当前文档于 2026 年 9 月检查。https://huggingface.co/docs/ Transformers/main/model_doc/llama

原题名与书目信息：[7] Hugging Face Transformers. Llama configuration documentation, including num_attention_heads and num_key_value_heads. Current documentation checked September 2026. https://huggingface.co/docs/ transformers/main/model_doc/llama

[8] Touvron, H. et al. Llama 2：开放基础和微调聊天模型（Models）。 2023.https://arxiv. org/abs/2307.09288

原题名与书目信息：[8] Touvron, H. et al. Llama 2: Open Foundation and Fine-Tuned Chat Models. 2023. https://arxiv. org/abs/2307.09288

[9] Jiang, A. Q. et al. 米斯特拉尔 7B。 2023.https://arxiv.org/abs/2310.06825

原题名与书目信息：[9] Jiang, A. Q. et al. Mistral 7B. 2023. https://arxiv.org/abs/2310.06825

[10] 元人工智能。介绍 Meta Llama 3. 2024. https://ai.meta.com/blog/meta-llama-3/

原题名与书目信息：[10] Meta AI. Introducing Meta Llama 3. 2024. https://ai.meta.com/blog/meta-llama-3/

[11] Dao, T. et al. FlashAttention：具有 IO 感知功能的快速、内存高效的精确注意力（Exact Attention）。NeurIPS 2022.https://arxiv.org/abs/2205.14135

原题名与书目信息：[11] Dao, T. et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. NeurIPS 2022. https://arxiv.org/abs/2205.14135

[12] Kwon, W. et al. 使用 PagedAttention 进行大型语言模型的高效内存管理。 SOSP 2023.https://arxiv.org/abs/2309.06180

原题名与书目信息：[12] Kwon, W. et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP 2023. https://arxiv.org/abs/2309.06180

源纪律。本手册中特定论文的实证结果作为引用实验的结果呈现，而不是普遍保证。框架行为具有明确的时间范围，可以更改。

参考图：针对哪个问题阅读什么内容

对应原稿第 40 页

问题	最佳起始来源
多头注意力从何而来？	Vaswani et al. [1]

瓦斯瓦尼等人。[1]

为什么提出MQA？ Shazeer [2] GQA 是如何定义的，以及 MHA 是如何转换的

安斯利等人。[3]

to it?

MLA 到底缓存了什么？ DeepSeek-V2 [4]，第 2.1 节 为什么 MLA 解耦 RoPE？ DeepSeek-V2 [4]，第 2.1.3 节 2.25 组和 93.3% 的数字来自哪里？

DeepSeek-V2 [4]

受控 MQA/GQA/MHA 和 MLA/MHA 消融报告了什么？

DeepSeek-V2 [4]，附录 D

当前的PyTorch如何暴露GQA？ PyTorch SDPA 文档 [6] 当前的 Llama 配置如何编码 KV 头计数？

抱脸骆驼文档 [7]

哪些 Llama 2 尺寸使用了 GQA？ Llama 2 报告 [8] Mistral 7B 中 GQA 在哪里使用？ Mistral 7B [9] FlashAttention 和 PagedAttention 与头部架构有何不同？

FlashAttention [11]，PagedAttention [12]

实施支持的变化速度比数学还要快。对于已部署的系统，将确切安装的框架、模型（Model）代码和注意内核（Kernel）文档视为支持的设备、数据类型、掩码、头维度、缓存布局和融合的 GQA/MLA 行为的权威。

面向人工智能工程师、开发人员和认真学习者的简短技术手册